

Single Agent Systems & Agent Pipelines

- Quiz

Instructions:

Answer the following questions. These questions are designed to test your understanding of agent systems, tools, and evaluation.

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

→ A stateful directed graph is a workflow where each step can store information from previous steps and use it later in the process. It consists of nodes connected by edges in the workflow that allows branching, looping, and decision-making based on the current state. This makes the system flexible and capable of handling complex tasks. Whereas A linear pipeline follows a fixed sequence of steps from start to finish without remembering previous states or changing its execution path. Stateful Paths adapt their workflow according to user inputs and intermediate results and therefore they are commonly used in Modern Ai agents.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

→ Edges are the connections between these nodes that determine how information flows from one task to another. Edges define the sequence in which these stages are connected and determine how the workflow progresses. For example, after identifying a mathematical query, the workflow is directed to the Calculator Tool through an edge. This structured design makes the workflow modular and easier to manage.

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

→ Conditional routing is a technique that enables an agent to determine the most suitable way to handle a user's request based on its intent. Instead of processing every query through the same workflow, the agent first identifies the type of request and then routes it to the appropriate tool. For example, queries related to calculations can be sent to the Calculator Tool, keyword extraction requests can be handled by the Keyword Extraction Tool, and all other queries can be directed to a General Response module. This improves the efficiency of the workflow, reduces unnecessary tool usage, and helps generate more accurate responses.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

→Cycles, or loops, allow an agent to repeat a particular task until it achieves the expected result or meets a specific condition. This helps the workflow continue instead of stopping because of temporary failures. For example, in our agent project, if the agent fails to identify whether the user wants to perform a calculation or extract keywords because of an unclear query, it can process the query again or ask for a clearer input before selecting a tool. This makes the agent more reliable and improves the overall user experience.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

→A single-agent system can perform the responsibilities of multiple agents by breaking a task into different stages. It first understands the user's request, then decides which tool is required, executes that tool, and finally prepares the response. Although only one agent is involved, it behaves like multiple specialized agents working together. In our project, the agent first detected the user's intent, then decided whether to use the Calculator Tool or the Keyword Extraction Tool, executed the selected tool, and finally returned the result in a structured JSON format.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

→JSON schema tools define a fixed structure that specifies how data should be passed between an agent and the tools it uses. It ensures that every input contains the required fields with the correct data type before the tool is executed. Similarly, the output also follows a predefined format, making it easier for the agent to understand and process the result. In our project, the agent returned responses in a structured JSON format containing fields like query, tool_used, result, and timestamp. This consistency reduces errors, improves communication between components, and makes the workflow easier to maintain.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

→Sequential tool calls are used when one task depends on the output of another, so each tool is executed in a specific order. In contrast, parallel tool calls allow multiple independent tasks to run at the same time, reducing the overall execution time. In our agent project, the workflow was sequential because the agent first had to detect the user's intent before deciding whether to use the Calculator Tool or the Keyword Extraction Tool. Parallel execution is more suitable when multiple tools can work independently without relying on each other's results, making the system faster and more efficient.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

→Error handling is the process of identifying and managing errors so that the agent can continue working without crashing. It improves the reliability of the system by handling unexpected situations gracefully. One strategy is using `try-except` blocks to catch exceptions and return meaningful error messages. Another strategy is validating the user's input before passing it to a tool, ensuring only valid requests are processed. In our project, if the user entered an invalid mathematical expression or an unsupported query, the agent returned an appropriate response instead of terminating the workflow. Logging such errors can also help developers debug and improve the agent over time.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

→Trajectory evaluation is the process of analyzing every step an agent takes while completing a task, instead of evaluating only the final result. It helps understand how the agent reached its answer by examining its decisions, tool selection, and execution flow. For example, if an agent is asked to summarize a document, trajectory evaluation checks whether it first retrieved the correct document, extracted the relevant content, and then generated the summary. Even if the final output looks correct, analyzing the complete workflow helps identify unnecessary steps, incorrect decisions, or inefficient tool usage, leading to better agent performance.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

→Task completion rate refers to the percentage of tasks that an agent successfully finishes without errors or manual intervention. Cost metrics measure the resources consumed while completing those tasks, such as execution time, API usage, memory, or computational cost. In a real-world application, these metrics can be monitored using system logs and performance reports over a large number of user requests. If the completion rate is low or the cost is high, developers can optimize the workflow by improving tool selection, eliminating unnecessary processing steps, and using more efficient algorithms. Maintaining a balance between a high completion rate and low operational cost is important for building a scalable and reliable agent system.